Guía para desarrollar lector CBR

https://gemini.google.com/app/ebd60e699929b1cf

User prompt: pues venga a empezar a construir

Response: ¡Manos a la obra! Vamos a estructurar la base de tu proyecto usando **Tauri v2 + React + TypeScript + Tailwind CSS**.

Para que el proyecto tenga calidad de producción desde el primer commit, usaremos un **Protocolo Personalizado en Tauri** (`comic://`). En lugar de convertir las imágenes a Base64 (que es lento y consume mucha RAM), React pedirá las páginas como si fueran URLs normales (`<img src="comic://localhost/..." />`) y Rust inyectará los bytes directamente desde el archivo comprimido en memoria.

1. Inicializar el proyecto y dependencias

Asegúrate de tener instalados **Node.js** y **Rust** (`rustup`). Abre tu terminal y ejecuta:

```
# 1. Crear el proyecto con la plantilla oficial de Tauri v2 (elige React + TypeScript + Vite)
npm create tauri-app@latest cover-clone -- --template react-ts
cd cover-clone

# 2. Instalar dependencias de UI y Estado en el Frontend
npm install zustand lucide-react clsx tailwind-merge
npm install -D tailwindcss postcss autoprefixer
npx tailwindcss init -p
```

Ahora, añade las librerías de Rust para leer archivos comprimidos y ordenar páginas de forma natural (`1.jpg, 2.jpg, 10.jpg`). Entra en la carpeta `src-tauri` y ejecuta:

```
cd src-tauri
cargo add zip natord serde_json
cargo add serde --features derive
# Nota: Empezamos con 'zip' (.cbz) porque compila sin dependencias externas de C++.
# Luego añadiremos 'unrar' para .cbr.
cd ..
```

2. Arquitectura de Carpetas (Clean Structure)

Organiza tu código así para que cualquiera que entre a tu GitHub vea una estructura modular y escalable:

```
cover-clone/
├── src/                        # FRONTEND (React + TS)
│   ├── components/
│   │   ├── library/            # Cuadrícula de portadas, estanterías
│   │   └── reader/             # Visor de páginas, controles de zoom, barra inferior
│   ├── hooks/                  # Ej: usePreloadPages.ts, useKeyboardShortcuts.ts
│   ├── store/                  # Estado global con Zustand (useComicStore.ts)
│   ├── types/                  # Interfaces TypeScript (Comic, Metadata, ReaderConfig)
│   └── App.tsx
├── src-tauri/                  # BACKEND NATIVO (Rust)
│   ├── src/
│   │   ├── archive/            # Lógica de descompresión (zip_reader.rs, rar_reader.rs)
│   │   ├── db/                 # Conexión y queries de SQLite
│   │   ├── commands.rs         # Funciones expuestas a React (@tauri::command)
│   │   └── lib.rs              # Configuración de Tauri y protocolo 'comic://'
│   └── Cargo.toml
└── .github/workflows/          # CI/CD para compilar releases automáticas
```

3. El Motor en Rust (`src-tauri/src/lib.rs`)

Este es el corazón de tu lector. Incluye un comando para listar las páginas ordenadas de un cómic y un comando para extraer los bytes de una página específica en memoria:

```
use std::fs::File;
use std::io::Read;
use serde::Serialize;
use zip::ZipArchive;

#[derive(Serialize)]
pub struct ComicInfo {
    path: String,
    pages: Vec<String>,
    total_pages: usize,
}

// Comprueba si el archivo dentro del ZIP/RAR es una imagen válida
fn is_image(filename: &str) -> bool {
    let lower = filename.to_lowercase();
    (lower.ends_with(".jpg") || lower.ends_with(".jpeg") || lower.ends_with(".png") || lower.ends_with(".webp"))
        && !lower.contains("__macosx")
}
```

```
// 1. Comando para abrir el cómic y obtener el índice de páginas ordenado
#[tauri::command]
fn open_comic(path: String) -> Result<ComicInfo, String> {
    let file = File::open(&path).map_err(|e| e.to_string())?;
    let mut archive = ZipArchive::new(file).map_err(|e| e.to_string())?;

    let mut pages: Vec<String> = Vec::new();

    for i in 0..archive.len() {
        if let Ok(file) = archive.by_index(i) {
            let name = file.name().to_string();
            if !file.is_dir() && is_image(&name) {
                pages.push(name);
            }
        }
    }

    // Ordenación natural: "Page 2.jpg" va antes que "Page 10.jpg"
    pages.sort_by(|a, b| natord::compare(a, b));
    let total_pages = pages.len();

    Ok(ComicInfo { path, pages, total_pages })
}

// 2. Comando para extraer una página concreta en memoria RAM (en crudo/bytes)
#[tauri::command]
fn get_page_bytes(comic_path: String, page_name: String) -> Result<Vec<u8>, String> {
    let file = File::open(&comic_path).map_err(|e| e.to_string())?;
    let mut archive = ZipArchive::new(file).map_err(|e| e.to_string())?;

    let mut page_file = archive.by_name(&page_name).map_err(|e| e.to_string())?;
    let mut buffer = Vec::with_capacity(page_file.size() as usize);
    page_file.read_to_end(&mut buffer).map_err(|e| e.to_string())?;

    Ok(buffer)
}

#[cfg_attr(mobile, tauri::mobile_entry_point)]
pub fn run() {
    tauri::Builder::default()
        .invoke_handler(tauri::generate_handler![open_comic, get_page_bytes])
        .run(tauri::generate_context!())
        .expect("Error al ejecutar la aplicación Tauri");
}
```

4. El Visor con Precarga en React (`src/components/reader/Reader.tsx`)

En el lado de TypeScript, convertimos esos bytes de Rust en un `Blob URL` ultrarrápido y precargamos la página siguiente en segundo plano:

```
import { useState, useEffect } from "react";
import { invoke } from "@tauri-apps/api/core";
import { ChevronLeft, ChevronRight } from "lucide-react";

interface ComicInfo {
  path: string;
  pages: string[];
  total_pages: number;
}

export function Reader({ comicPath }: { comicPath: string }) {
  const [comic, setComic] = useState<ComicInfo | null>(null);
  const [currentIndex, setCurrentIndex] = useState(0);
  const [imageUrl, setImageUrl] = useState<string>("");
  const [pageCache, setPageCache] = useState<Record<number, string>>({});

  // 1. Cargar la lista de páginas al abrir el cómic
  useEffect(() => {
    invoke<ComicInfo>("open_comic", { path: comicPath })
      .then((data) => {
        setComic(data);
        setCurrentIndex(0);
      })
      .catch(console.error);
  }, [comicPath]);

  // Función auxiliar para pedir bytes a Rust y crear una URL en memoria
  const loadPageToBlob = async (index: number, currentComic: ComicInfo) => {
    if (pageCache[index] || index < 0 || index >= currentComic.total_pages) return;
```

```
  const bytes = await invoke<number[]>("get_page_bytes", {
    comicPath: currentComic.path,
    pageName: currentComic.pages[index],
  });

  const uint8Array = new Uint8Array(bytes);
  const blob = new Blob([uint8Array]);
  const url = URL.createObjectURL(blob);

  setPageCache((prev) => ({ ...prev, [index]: url }));
  return url;
};

// 2. Actualizar la página actual y precargar la siguiente (Pre-fetching)
useEffect(() => {
  if (!comic) return;

  const updateView = async () => {
    if (pageCache[currentIndex]) {
      setImageUrl(pageCache[currentIndex]);
    } else {
      const url = await loadPageToBlob(currentIndex, comic);
      if (url) setImageUrl(url);
    }
    // Precargar en segundo plano las 2 páginas siguientes
    loadPageToBlob(currentIndex + 1, comic);
    loadPageToBlob(currentIndex + 2, comic);
  };

  updateView();
}, [currentIndex, comic]);

// 3. Navegación con las flechas del teclado
useEffect(() => {
  const handleKeyDown = (e: KeyboardEvent) => {
    if (!comic) return;
    if (e.key === "ArrowRight" && currentIndex < comic.total_pages - 1) {
      setCurrentIndex((prev) => prev + 1);
    } else if (e.key === "ArrowLeft" && currentIndex > 0) {
      setCurrentIndex((prev) => prev - 1);
    }
  };
  window.addEventListener("keydown", handleKeyDown);
  return () => window.removeEventListener("keydown", handleKeyDown);
}, [currentIndex, comic]);

if (!comic) return <div className="text-white p-10">Abriendo cómic...</div>;

return (
  <div className="relative flex items-center justify-center w-screen h-screen bg-neutral-950 select-none overflow-
    {/* Imagen de la página actual */}
    {imageUrl && (
      <img
        src={imageUrl}
        alt={`Página ${currentIndex + 1}`}
        className="max-h-full max-w-full object-contain transition-opacity duration-150"
      />
    )}

    {/* Barra flotante inferior estilo Cover */}
    <div className="absolute bottom-6 flex items-center gap-4 bg-neutral-900/80 backdrop-blur-md px-6 py-3 rounded
      <button
        onClick={() => setCurrentIndex((p) => Math.max(0, p - 1))}
        disabled={currentIndex === 0}
        className="hover:text-amber-400 disabled:opacity-30"
      >
        <ChevronLeft size={24} />
      </button>

      <span className="text-sm font-medium tracking-wide min-w-[80px] text-center">
        {currentIndex + 1} / {comic.total_pages}
      </span>

      <button
        onClick={() => setCurrentIndex((p) => Math.min(comic.total_pages - 1, p + 1))}
        disabled={currentIndex === comic.total_pages - 1}
        className="hover:text-amber-400 disabled:opacity-30"
      >
```

```
          <ChevronRight size={24} />
        </button>
      </div>
    </div>
  );
}
```

¿Cuál es el siguiente paso inmediato?

1. Ejecuta `npm run tauri dev` para comprobar que la ventana nativa compila y se abre.
2. Añadir el plugin de diálogo de archivos de Tauri (`npm run tauri add dialog`) para poder pulsar un botón "Abrir Cómic", elegir un archivo `.cbz` de tu PC y pasárselo al componente `<Reader/>`.

¿Quieres que configuremos ahora el selector de archivos y el soporte para `.cbr` (RAR), o prefieres diseñar primero la pantalla de la **Estantería/Biblioteca** estilo Cover?

User prompt: Tema de las carátulas como se gestiona vienen como meta datos en los propios archivos CBR cbz o con una API o como

Response: En aplicaciones como **Cover**, la gestión de carátulas sigue una estrategia híbrida en cascada para ser rápida y no depender obligatoriamente de internet.

El flujo estándar de la industria funciona así:

1. El estándar por defecto: La primera imagen del archivo (Offline)

En el 95% de los archivos `.cbz` y `.cbr`, **no hay una carátula separada como metadato especial**: la carátula es sencillamente el **primer archivo de imagen** que aparece dentro del archivo comprimido al ordenarlos alfabéticamente (por ejemplo, `000_cover.jpg`, `01.jpg`, etc.).

Cómo lo hace Cover:

1. Cuando añades una carpeta a la biblioteca, un hilo en segundo plano abre el archivo comprimido.
2. Lee únicamente el índice del ZIP o RAR (sin descomprimir el cómic entero).
3. Busca el primer archivo de imagen válido.
4. Descomprime **únicamente esa imagen** en memoria.
5. La redimensiona a un tamaño ligero (por ejemplo, 300×450 px en formato WebP o JPEG).
6. Guarda esa miniatura en una carpeta de caché local de la app (ej. `AppData/Local/Cover/thumbnails/{hash_del_comic}.webp`).
7. Guarda la ruta de esa miniatura en SQLite para mostrarla al instante cuando abres la app.

2. Metadatos internos: `ComicInfo.xml` (Offline)

Muchos archivos `.cbz`/`.cbr` (especialmente si provienen de gestores como ComicRack, calibre o escaneos organizados) incluyen un archivo en su raíz llamado **`ComicInfo.xml`**.

Dentro de ese XML no suele venir la imagen incrustada, pero sí metadatos enriquecidos:

```
<ComicInfo>
  <Title>The Dark Knight Returns</Title>
  <Series>Batman</Series>
  <Number>1</Number>
  <Summary>Sinopsis del cómic...</Summary>
  <Writer>Frank Miller</Writer>
  <Pages>
    <!-- Algunos especifican explícitamente cuál es la portada -->
    <Page Image="0" Type="FrontCover" />
    <Page Image="1" Type="Story" />
    <Page Image="23" Type="BackCover" />
  </Pages>
</ComicInfo>
```

Si el XML contiene la etiqueta `<Page Type="FrontCover"/>`, la aplicación sabe con exactitud qué página usar como carátula (por si acaso la primera página fuera una hoja en blanco o créditos). Además, de aquí extraes título, autor, año y sinopsis automáticamente sin conexión.

3. Scraping con APIs externas (Online / Opcional)

Si el archivo se llama algo genérico como `Batman 001 (1986).cbr` y no tiene `ComicInfo.xml`, las aplicaciones avanzadas permiten conectar con APIs públicas de cómics para enriquecer la ficha:

- **Comic Vine API:** Es la base de datos de cómics más grande y el estándar que usan casi todos los gestores. Te da la carátula oficial en altísima resolución, lista de personajes, equipo creativo, fecha de publicación, etc.
- **AniList / MyAnimeList API / MangaDex:** Muy populares si la app se enfoca más en Manga.

Cómo se implementa:

1. Parseas el nombre del archivo con expresiones regulares para extraer título y número (ej. regex para `"Batman"` y `"001"`).
2. Consultas la API (`GET [https://comicvine.gamespot.com/api/search/?query=Batman] (https://comicvine.gamespot.com/api/search/?query=Batman) 1`).
3. Descargas la portada oficial y sobrescribes la miniatura generada localmente.

¿Cuál es la arquitectura recomendada para tu proyecto?

Para que tu aplicación se sienta **instantánea** como Cover, implementa este orden de prioridad:

```
¿El archivo tiene ComicInfo.xml?
   ├── SÍ ──> ¿Define <Page Type="FrontCover"/>? ──> Extraer esa página a caché
   └── NO ──> Extraer la 1ª imagen alfabética (.jpg/.png) a caché

(Opcional / Manual) ──> Botón "Buscar metadatos en Comic Vine" para descargar la carátula oficial.
```

De esta forma, la biblioteca del usuario carga sus cientos de cómics en pocos segundos nada más instalar la app, sin necesidad de conexión ni de que el usuario configure claves de API.

User prompt: Olvidadate de intentar copiar a cover eso era solo para que entendieses el tipo de aplicación que quería hacer

Response: Entendido perfectamente. Quitemos a Cover de la ecuación y centrémonos en crear **tu propio lector y gestor de cómics**, adaptado a lo que a ti te interese construir y a lo que mejor hable de ti como desarrollador.

Al no estar atado al diseño o flujo de otra app, tienes total libertad para decidir el enfoque. Hay varias direcciones interesantes que puedes tomar según lo que más te motive programar:

1. El enfoque "Minimalista y Ultrarrápido"

- **Propósito:** Abrir cualquier archivo (CBR, CBZ, PDF, EPUB de imagen) al instante sin importar si pesa 500 MB.
- **Características clave:**
  - Cero configuración previa. Puedes arrastrar un archivo o carpeta y empezar a leer.
  - Modos de lectura fluidos: cascada infinita vertical (estilo *webtoon*), doble página con detección automática de orientaciones, o lectura horizontal derecha-a-izquierda (manga).
  - Foco en atajos de teclado, gestos táctiles y una interfaz limpia que desaparece por completo mientras lees.

2. El enfoque "Gestor de Biblioteca Inteligente"

- **Propósito:** Para gente que tiene cientos de gigas en cómics/mangas y necesita orden.
- **Características clave:**
  - Escaneo de carpetas en segundo plano con base de datos local (SQLite).
  - Agrupación automática por sagas, tomos, universos o editoriales leyendo metadatos (`ComicInfo.xml`) o parseando nombres de archivo.
  - Filtros avanzados: "No leídos", "En progreso", cómics favoritos, tags personalizados.
  - Estadísticas de lectura (tiempo dedicado, páginas leídas por semana, etc.).

3. El enfoque con "Funciones Diferenciales" (muy potente para GitHub)

- **Traducción o lectura asistida:** OCR local para reconocer texto en viñetas o permitir seleccionar texto.
- **Detección inteligente de viñetas:** Navegación guiada viñeta a viñeta (haciendo zoom automático en cada viñeta en lugar de ver la página completa).
- **Soporte cliente-servidor:** Conexión con servidores personales mediante protocolos estándar como OPDS (Komga, Kavita) o sincronización con servicios en la nube.

¿Hacia dónde te gustaría enfocarlo?

Para empezar a dar forma a **tu** diseño y arquitectura:

1. **¿Qué tipo de lector te gustaría que fuera?** (¿Algo centrado principalmente en el visor de lectura puro y duro, o te interesa mucho la parte de organizar la estantería y biblioteca?).
2. **¿Qué funciones te gustaría que tuviera que normalmente echas en falta en otros lectores?**

Dime qué visión te llama más la atención y definimos las bases técnicas y los módulos a programar.

User prompt: Tema docker cuvernetes nos podría ahorrar el tema de nmp y hacer que fuese mejor de rendimiento?

Response: La respuesta corta es: **no, para una aplicación de escritorio como esta, Docker y Kubernetes no te van a ahorrar npm ni van a mejorar el rendimiento; de hecho, harían todo lo contrario.**

Conviene aclarar qué papel cumple cada tecnología para ver por qué no encajan aquí y qué alternativas sí resuelven tus dudas sobre npm y rendimiento.

1. ¿Por qué Docker y Kubernetes no tienen sentido aquí?

- **Kubernetes (K8s)** es un orquestador para coordinar cientos o miles de contenedores distribuidos en clústeres de servidores en la nube (backend web, microservicios de empresas como Netflix o Uber). En una app que corre en el ordenador de un usuario final, Kubernetes no tiene ninguna cabida.
- **Docker** sirve para empaquetar aplicaciones de servidor para que corran igual en cualquier máquina Linux/Cloud.
    - **No mejora el rendimiento:** Al contrario, añade una capa de virtualización/aislamiento. En sistemas como Windows o macOS, correr Docker implica levantar una máquina virtual Linux por debajo, consumiendo gigabytes de RAM solo para existir.
    - **No ahorra npm:** Para construir el código frontend dentro del contenedor, Docker tendría que ejecutar exactamente los mismos comandos (`npm install`, `npm run build`), solo que dentro de la imagen.
    - **Experiencia de usuario inviable:** No puedes pedirle a un usuario que quiera leer un cómic en su PC que tenga Docker Desktop instalado y corriendo para abrir un visor.

2. ¿Cómo deshacerte de los problemas de `npm`?

Si lo que te preocupa de `npm` es que es lento, descarga gigabytes en la carpeta `node_modules` o resulta pesado, tienes dos opciones mucho mejores:

Opción A: Reemplazar `npm` por herramientas ultrarrápidas

Hoy en día casi nadie en proyectos punteros usa `npm` a secas. Puedes usar:

- **pnpm:** Es rapidísimo y no duplica paquetes en disco. Si 10 proyectos usan React, solo se descarga una vez en tu disco duro en un almacén global.
- **Bun:** Está escrito en Zig/C++. Instala dependencias hasta 20 veces más rápido que `npm` y puede sustituir por completo a Node.js en el entorno de desarrollo.

Opción B: Eliminar JavaScript y `npm` por completo (100% Nativo)

Si no quieres tocar `node_modules` ni saber nada de Node/npm, la alternativa reina en rendimiento es **no usar tecnologías web**:

1. **Rust puro (con librerías de UI nativas como Slint, Iced o egui):**
    - Cero `npm`, cero Node.js.
    - La UI y el motor de lectura se compilan en un único archivo ejecutable `.exe` (o binario en Mac/Linux) de apenas 15–30 MB.
    - Rendimiento y consumo de RAM imbatibles (menos de 50 MB de RAM en ejecución).
2. **C# con WinUI 3 o Avalonia UI (.NET 8/9):**
    - El gestor de paquetes es `NuGet` (integrado en el SDK de .NET).
    - Rápido, componentes visuales listos para usar y excelente aceleración por GPU con DirectX/Skia.
3. **C++ / Qt:**
    - El estándar clásico de visores de escritorio de alto rendimiento, aunque con una curva de aprendizaje más pronunciada.

3. ¿Cómo se entrega el proyecto al usuario final?

Ten en cuenta que, aunque uses **Tauri con React/TypeScript**:

- **El usuario final NUNCA necesita `npm` ni Node.js.**
- Cuando compilas el proyecto para producción (`npm run tauri build`), Tauri empaqueta el frontend precompilado (HTML/JS estático) junto con el binario de Rust en un único instalador nativo (`.msi`, `.exe`, `.dmg` o `.deb`).
- El usuario simplemente hace doble clic, instala la app y la usa de forma nativa. `npm` solo existe en tu ordenador mientras programas.

Resumen y recomendación

- **Descartar:** Docker y Kubernetes (son herramientas de infraestructura de servidores/cloud, no de apps de escritorio).
- **Si te gusta la flexibilidad de diseñar interfaces con CSS/React:** Usa **pnpm** o **Bun** en lugar de `npm` para que la instalación de paquetes sea casi instantánea, y mantén Tauri.
- **Si quieres eliminar dependencias web y buscar el máximo rendimiento técnico en GitHub:** Desarrolla la aplicación **100% en Rust** (usando por ejemplo *Slint UI* o *Iced*) o en **C# / Avalonia**.

User prompt: Tema de las carátulas para elegir un comic o novela es posible que el diseño sea 3D?

Response: **Sí, es totalmente posible** y es una de las características que más impacto visual generan en un proyecto para GitHub. Le da una presencia diferencial frente a las típicas interfaces planas de cuadrícula.

Dependiendo de qué tan inmersivo lo quieras, existen tres formas de implementarlo:

1. Carrusel 3D o "Cover Flow"

Inspirado en el mítico efecto Cover Flow de Apple o en interfaces interactivas de consolas. Las carátulas están dispuestas a lo largo de un eje Z con rotación y profundidad en perspectiva.

- **Cómo interactúa el usuario:** Te desplazas horizontalmente con la rueda del ratón o las flechas del teclado. El cómic central se coloca de frente en primer plano y las portadas adyacentes se inclinan lateralmente con reflejos y sombras.
- **Por qué funciona bien:** Es muy vistoso, intuitivo para navegar por colecciones o sagas concretas y no sobrecarga la vista.

2. Modelado de libro/cómic 3D con grosor y lomo

En lugar de mostrar una imagen plana rectangular, cada cómic se renderiza como una caja 3D (*box mesh*) con textura frontal, grosor personalizable y textura de lomo y páginas.

- **Efecto de rotación al pasar el cursor (Hover Tilt):** Al mover el ratón sobre un tomo, el libro rota suavemente en 3D siguiendo la posición del cursor, mostrando el grosor del volumen y un brillo dinámico.
- **Grosor dinámico:** Puedes calcular la profundidad del modelo 3D en función del número real de páginas del archivo (`total_pages`). Un manga de 200 páginas se verá más delgado que un recopilatorio *Omnibus* de 800 páginas.
- **Apertura de portada:** Al hacer clic para leer, la portada frontal puede rotar sobre su eje como si abrieras físicamente la tapa del cómic antes de entrar al visor de lectura.

3. Estantería física virtual (Virtual Bookshelf)

Renderizar una estantería de madera o metal en 3D donde los cómics descansan de canto (mostrando el lomo) o apoyados en atriles. Al pasar el cursor por encima, el cómic sobresale un poco hacia adelante de la balda.

¿Cómo se implementa técnicamente?

Tienes dos alternativas principales según la complejidad que busques:

Enfoque	Tecnologías	Rendimiento	Dificultad
Puro CSS 3D (Transforms)	`perspective`, `transform: rotateY()`, `preserve-3d`	Excelente (GPU nativa del navegador web)	Baja. No necesitas librerías de 3D pesadas.
WebGL con Three.js / React Three Fiber	`Three.js` o `@react-three/fiber` + shaders	Alto (permite iluminación dinámica, reflejos y sombras reales)	Media. Muy llamativo a nivel de ingeniería gráfica.

La opción más limpia si usas React (CSS 3D sin dependencias):

Puedes crear el efecto de volumen y giro 3D únicamente con **Tailwind CSS**, mapeando la primera imagen en la cara delantera (`front`), un degradado blanco/grisáceo para el lomo (`spine`) y páginas laterales (`side`):

```
// Ejemplo de tarjeta con inclinación 3D en hover
<div className="group [perspective:1000px]">
  <div className="w-52 h-72 relative transition-transform duration-500 [transform-style:preserve-3d] group-hover:[tr
    {/* Portada */}
    <img
      src={coverUrl}
      className="w-full h-full object-cover rounded-r-sm shadow-2xl"
      alt="Carátula"
    />
    {/* Efecto de lomo/grosor lateral */}
    <div className="absolute top-0 right-0 w-4 h-full bg-neutral-200 [transform:rotateY(90deg)_translateZ(2px)] orig
  </div>
</div>
```

Si prefieres una experiencia 3D completa donde puedas orbitar con la cámara alrededor del libro con iluminación realista, **React Three Fiber** (`r3f`) es el estándar moderno en React.

User prompt: Seguiríamos usando rust?

Response: **Sí, absolutamente.** Mantener Rust no solo es compatible con el diseño 3D, sino que es la combinación perfecta por cómo se reparten las tareas:

¿Cómo encaja Rust con una interfaz 3D?

En una arquitectura con **Tauri**, la interfaz visual y el motor del sistema van desacoplados:

```
[ FRONTEND: React + Three.js / WebGL / CSS 3D ]
   │
   │  - Dibuja el modelo 3D del libro/cómic.
   │  - Controla la rotación, sombras, luces e inclinación.
   │  - Pide la portada: "¿Tienes la miniatura de Batman #1?"
   ▼
[ BACKEND: Rust ]
   │
   │  - Lee el archivo .cbr / .cbz en milisegundos.
   │  - Extrae y redimensiona la imagen de la portada.
   │  - Lee el ComicInfo.xml (número de páginas, autor, etc.).
   │  - Gestiona la base de datos SQLite y la caché de disco.
   ▼
[ Devuelve los bytes directamente a la GPU / Canvas 3D ]
```

¿Por qué mantener Rust aquí?

1. **Rendimiento de carga para texturas 3D:** Renderizar objetos en 3D en la pantalla requiere que las imágenes (texturas) carguen de inmediato. Si el backend fuera lento, verías modelos 3D en blanco o "parpadeos" mientras giran. Rust descomprime las portadas y calcula los metadatos a velocidad nativa.
2. **El "grosor" del libro 3D depende de Rust:** Rust lee cuántas páginas reales tiene el cómic (`total_pages`) y se lo envía al frontend. Con ese dato exacto, el modelo 3D ajusta automáticamente su grosor (un tomo de 600 páginas se ve tres veces más ancho que una grapa de 24 páginas).
3. **Tu perfil en GitHub:** Mantienes la combinación más valorada:
    - **Gráficos e interacción moderna:** React + Three.js / WebGL / CSS 3D.
    - **Ingeniería de sistemas y bajo nivel:** Rust (I/O ultrarrápido, multihilo, memoria segura).

En resumen: **el 3D vive en el frontend** (acelerado directamente por la tarjeta gráfica / GPU de la máquina del usuario) y **Rust se encarga de alimentarlo con datos e imágenes sin despeinarse**.

User prompt: Vale en cuanto a la función ocr para buscar información o traducir me gustaría dar la opción de poner una key por si el usuario tiene algún tipo de subscripción o utilizar un modelo local yo por ejemplo suelo usar ollama no sé si hay más proveedores

Response: Para implementar esta función de manera limpia y profesional, lo ideal es adoptar un enfoque **multimodal (Vision LLM)**: en lugar de separar el OCR tradicional (como Tesseract) y la traducción en dos pasos torpes, el usuario selecciona una viñeta o área de la página, se recorta esa imagen y se envía directamente al modelo con un prompt como:

> *"Transcribe el texto visible en los bocadillos de este cómic, tradúcelo al español y añade notas de contexto cultural o referencias si las hay."*

Dado que la gran mayoría de servicios (tanto locales como en la nube) han adoptado el formato de **API compatible con OpenAI**, puedes soportar múltiples proveedores usando prácticamente el mismo cliente HTTP en Rust.

1. Proveedores Locales (Privacidad y Cero Costes)

- **Ollama (Tu opción actual):**
    - **Cómo funciona:** Corre en `http://localhost:11434`. Soporta la API estándar (`/v1/chat/completions`) y su API nativa.
    - **Modelos de visión recomendados:** `llama3.2-vision` (11B o 90B) y `minicpm-v` (excelente relación calidad/peso para OCR en cómics y lectura de texto denso).
    - **Ventaja:** Cero configuración adicional si ya lo tienes instalado; tu lector solo comprueba si el puerto 11434 responde.
- **LM Studio:**
    - Muy extendido entre usuarios de escritorio que prefieren una interfaz gráfica para descargar modelos de Hugging Face.
    - Levanta un servidor local en `http://localhost:1234/v1` exactamente compatible con OpenAI.
- **Jan.ai / LocalAI:**
    - Alternativas de código abierto que también exponen endpoints locales compatibles con OpenAI.
- **OCR Clásico Especializado (Manga-OCR):**
    - Si el usuario lee manga japonés (texto vertical con kanjis complejos), los LLM locales a veces inventan caracteres. `manga-ocr` es una red neuronal ligera especializada en texto japonés de viñetas que se puede empaquetar o consultar localmente.

2. Proveedores Cloud ("Bring Your Own Key")

Para usuarios que no tienen una tarjeta gráfica potente para correr modelos de visión locales:

Proveedor	Modelo Recomendado	Ventajas para Cómics
Google Gemini	`gemini-1.5-flash`	Relación calidad-precio imbatible y tiempos de respuesta mínimos procesando imágenes.
OpenAI	`gpt-4o-mini` o `gpt-4o`	Estándar de la industria, reconocimiento impecable de onomatopeyas y jerga.
Anthropic	`claude-3-5-sonnet`	La mejor calidad en traducción literaria y adaptación de diálogos con doble sentido.
Groq	`llama-3.2-11b-vision-preview`	Inferencia ultrarrápida (casi instantánea), ideal si el usuario quiere traducción en tiempo real.
OpenRouter	Cualquier modelo	El mejor agregador: el usuario recarga saldo en una sola cuenta y tu app puede usar modelos de OpenAI, Anthropic, Meta o Mistral con una sola API key.
DeepL API	`DeepL API Free / Pro`	Solo traduce texto (requiere un OCR previo), pero sigue siendo el estándar en traducción precisa sin intervención de LLMs.

3. Arquitectura del Selector en tu App

Para que el repositorio tenga una arquitectura limpia y extensible, define en TypeScript y Rust una estructura polimórfica:

```
export type AIProvider = "ollama" | "lmstudio" | "openai" | "gemini" | "anthropic" | "openrouter" | "custom";

export interface AISettings {
  provider: AIProvider;
  endpointUrl: string; // ej: "http://localhost:11434/v1" o la URL de OpenAI
  apiKey?: string;     // Vacío si es Ollama o LM Studio
  model: string;       // ej: "llama3.2-vision", "gpt-4o-mini", etc.
  targetLanguage: string; // ej: "es", "en", "fr"
}
```

Flujo de la interacción en la interfaz:

```
[ Usuario pulsa botón 'Traducir / OCR' en el visor ]
                        │
                        ▼
[ Herramienta de selección de área (Lasso / Rectángulo) sobre la viñeta ]
                        │
                        ▼
[ Rust recorta las coordenadas de la imagen original en alta resolución ]
                        │
                        ▼
[ Rust envía petición HTTP POST con la imagen en Base64 al Endpoint configurado ]
   ├── Si es Ollama: http://localhost:11434/v1/chat/completions (sin Auth)
   └── Si es Cloud:  api.openai.com / openrouter.ai (con Bearer Token)
                        │
                        ▼
[ Ventana emergente (Tooltip) sobre la viñeta con el texto transcrito y traducido ]
```

4. ¿Por qué delegar esta llamada a Rust y no a React?

1. **Evitas problemas de CORS:** Hacer peticiones desde la vista web de Tauri a `localhost:11434` o a APIs externas puede disparar restricciones de origen cruzado en el navegador; Rust hace peticiones HTTP nativas directas con la librería `reqwest` sin restricciones de CORS.
2. **Seguridad de las API Keys:** Las claves del usuario se pueden almacenar de forma segura usando el gestor de credenciales del sistema operativo (Keychain en macOS, Credential Manager en Windows) mediante librerías de Rust como `keyring-rs`, en lugar de dejarlas en texto plano en el `localStorage` del navegador.
3. **Procesamiento de imagen nativo:** El recorte del recuadro seleccionado por el usuario sobre la página original en alta resolución se hace directamente en Rust con la librería `image`, sin penalizar el rendimiento del hilo principal de la interfaz gráfica.

User prompt: Revisa la conversación e investiga literatura y ejemplos existentes y dime qué opinas sobre las decisiones que se han tomado

Response: Al revisar las decisiones tomadas hasta ahora y contrastarlas con proyectos reales de código abierto y literatura técnica sobre visores gráficos y de cómics, el balance es muy positivo: **la arquitectura general es sobresaliente para un portafolio moderno, pero hay tres trampas técnicas concretas en lo que hemos hablado que conviene corregir antes de escribir más código.**

1. Lo que está muy bien decidido (Aciertos)

- **Descartar Docker/Kubernetes para el cliente:** Es la decisión correcta. Ningún lector de escritorio (como *YACReader*, *SumatraPDF* o *Thorium Reader*) usa contenedores para distribuirse al usuario final.
- **El Stack Tauri v2 (Rust + React/TypeScript):** Proyectos modernos de alto rendimiento (como *Spacedrive* o *Lapce*) han demostrado que delegar el manejo de archivos e imágenes pesadas a Rust y dejar la interfaz a tecnologías web modernas ofrece el mejor equilibrio entre

velocidad de desarrollo y bajo consumo de RAM.

- **Arquitectura de IA unificada (BYOK + Ollama):** Es exactamente el patrón que usan hoy herramientas líderes como *Obsidian*, *Zed* o *Continue.dev*. Soportar el estándar de API de OpenAI te permite conectar con Ollama, LM Studio, OpenRouter y proveedores cloud con una sola implementación en `reqwest` (Rust).

2. Correcciones importantes respecto a lo que hablamos antes

Al comparar el código inicial que te propuse con cómo funcionan por dentro lectores consolidados como **MComix**, **CDisplayEx**, **OpenComic** o **Komga**, hay tres puntos que debemos rectificar:

A. Error en el código inicial: `invoke` vs. Custom Protocol (`comic://`)

En mi primer ejemplo de código usé `invoke("get_page_bytes")` devolviendo un `Vec<u8>`. **En la práctica, esto es un cuello de botella grave en Tauri.**

- **Por qué falla:** `invoke` serializa el array de bytes a formato JSON (`[255, 216, 255...]`) para cruzar el puente entre Rust y JavaScript. Una imagen de 4 MB se convierte en un texto JSON gigantesco que congela la interfaz unos milisegundos al pasar de página.
- **Cómo lo hace la literatura/industria en Tauri:** Se usa `register_uri_scheme_protocol` (en Tauri v2). Rust levanta un servidor interno virtual y React pide `<img src="comic://localhost/page/1" />`. Los bytes viajan en crudo (binario puro) directamente al motor de renderizado de imágenes del navegador sin pasar por JavaScript ni JSON.

B. La trampa de los archivos `.cbr` (RAR5 y Licencias)

Los archivos `.cbz` (ZIP) son triviales porque el algoritmo Deflate es libre y permite acceso aleatorio instantáneo a cualquier página. Con los `.cbr` (RAR) hay dos problemas clásicos documentados en todos los lectores open-source:

1. **RAR4 vs RAR5:** Muchas librerías de Rust puras solo leen formatos antiguos de RAR. Para leer archivos `.cbr` modernos (RAR5), tendrás que enlazar con el código C++ oficial de `unrar` (usando el crate `unrar` en Rust) o con `libarchive`.
2. **Archivos RAR "Sólidos" (Solid Archives):** Algunos `.cbr` están comprimidos como un bloque continuo. Si pides la página 50 sin haber descomprimido las 49 anteriores, el descompresor tarda muchísimo. Los lectores como *YACReader* o *SumatraPDF* detectan si un RAR es "sólido" o muy lento y, solo en ese caso, lo descomprimen en segundo plano en un buffer temporal de memoria o caché.

C. El problema del OCR con Vision LLMs en Manga/Cómic

Usar directamente modelos multimodales (como `llama3.2-vision` en Ollama) sobre un recorte de viñeta funciona genial en cómics americanos o europeos (texto horizontal en alfabeto latino). Sin embargo, la literatura de proyectos específicos como **Poricom**, **BallonsTranslator** o **Mokuro** demuestra que:

- Los LLMs de visión generales (sobre todo los locales de 7B–11B parámetros) **alucinan mucho con el texto vertical japonés** o con fuentes tipográficas muy estilizadas.
- **La arquitectura estándar real en estos proyectos es un Pipeline de 2 fases:**
    1. **Fase 1 (OCR ligero especializado o general):** Extraer el texto puro (usando las APIs nativas del sistema operativo como *Windows Media OCR* / *Apple Vision*, o *PaddleOCR* / *Manga-OCR*).
    2. **Fase 2 (LLM solo de texto o Vision LLM):** Pasarle a Ollama/OpenAI la imagen **más** el texto detectado por el OCR para que traduzca con precisión y explique el contexto sin inventarse palabras.

3. Sobre el diseño 3D de las carátulas (Realismo vs. Usabilidad)

Revisando experiencias previas (como el antiguo *Cover Flow* de Apple o lectores con estanterías 3D):

Enfoque 3D	Veredicto práctico
Escenario 3D completo (WebGL / Three.js en toda la biblioteca)	**Peligroso.** Cuando un usuario tiene 500 cómics, renderizar un canvas 3D para toda la cuadrícula dispara el uso de GPU, ventila el portátil al máximo y hace que buscar un cómic sea más lento que en una cuadrícula 2D.
Tarjetas con inclinación 3D en CSS + Vista de detalle en 3D	**El punto dulce.** La biblioteca carga instantáneamente como una cuadrícula limpia con efectos de volumen/grosor en CSS al pasar el ratón (como las cartas de *Marvel Snap* o *Steam*), y cuando haces clic en un cómic para ver su ficha/sinopsis, ahí sí muestras el modelo 3D interactivo del tomo completo con su lomo.

Veredicto Final de Arquitectura Recomendada

Si quieres que un ingeniero Senior vea tu repositorio en GitHub y piense *"esta persona sabe exactamente lo que hace"*, esta es la hoja de ruta depurada:

1. **Comunicación Rust-React:** Reemplazar `invoke` de imágenes por el protocolo binario `register_uri_scheme_protocol` (`comic://`).
2. **Motor de Archivos (Trait `ComicArchive` en Rust):** Crear una interfaz común en Rust que implemente `ZipReader` (para CBZ) y `RarReader` (para CBR), manteniendo el archivo abierto en un `Mutex` o `RwLock` en el estado de Tauri para no abrir y cerrar el archivo del disco en cada página.
3. **UI 3D Híbrida:** CSS 3D acelerado por hardware para la biblioteca general (grosor dinámico según páginas) y un visor de tomo 3D detallado al seleccionar un cómic.

4. **Módulo IA/OCR:** Soportar Vision LLM directo (recorte de viñeta a Ollama/OpenAI), dejando la puerta abierta a usar el OCR nativo del sistema operativo como apoyo.

User prompt: Es realmente rust más rápido que java o Python u otros lenguajes?

Response: **Sí, Rust es objetivamente más rápido y consume mucha menos memoria que Python y Java**, pero la magnitud de la diferencia depende de con cuál lo compares y de qué tarea estemos hablando (descomprimir un archivo, redimensionar imágenes o gestionar memoria).

1. Comparativa de Rendimiento y Recursos

Los lenguajes de programación actuales se dividen en tres categorías según cómo se ejecutan en el procesador:

Lenguaje	Categoría	Velocidad de CPU (Relativa)	Consumo de RAM	Gestión de Memoria
Rust / C / C++	Compilados a código máquina nativo (AOT)	**1x** (Máxima velocidad)	**Mínimo** (15–40 MB base)	En tiempo de compilación (*Ownership* en Rust / Manual en C++)
Java / C# / Go	Compilados con Máquina Virtual / JIT	**1.2x – 2.5x** más lentos	**Alto** (150–400 MB base)	Recolector de Basura (*Garbage Collector*)
Python / JS	Interpretados / Dinámicos	**10x – 100x** más lentos (en código puro)	**Medio-Alto**	Recolector de Basura + Conteo de referencias

2. ¿Por qué existe esta diferencia en tu lector de cómics?

Rust vs. Python (La diferencia es abismal)

- **Cómo funciona Python:** Es un lenguaje interpretado y de tipado dinámico. Cada vez que Python lee un byte o ejecuta un bucle, tiene que comprobar en tiempo de ejecución de qué tipo es la variable. En cálculo puro, Rust suele ser entre **20 y 100 veces más rápido** que Python.
- **El "truco" de Python:** Cuando usas librerías famosas en Python (como *Pillow* para imágenes o *NumPy*), esas librerías en realidad están escritas por debajo en **C, C++ o Rust**. Python solo actúa como "pegamento".
- **El problema de Python en escritorio:** Empaquetar una app de Python para que un usuario la instale en Windows o Mac obliga a meter todo el intérprete de Python dentro del ejecutable (usando *PyInstaller*), generando archivos pesados, arranques lentos y dificultades con el multihilo real.

Rust vs. Java / C# (El secreto está en la RAM y las pausas)

Java es sorprendentemente rápido gracias a su compilador **JIT (Just-In-Time)**, que optimiza el código mientras el programa corre. En operaciones matemáticas largas, Java se acerca bastante a Rust. Sin embargo, para un lector de cómics con imágenes de alta resolución, Rust gana por dos motivos críticos:

1. **Ausencia de Recolector de Basura (Garbage Collector - GC):**
    - En **Java**, cuando pasas 20 páginas de un cómic (4 MB por imagen), las páginas viejas quedan en la memoria RAM esperando a que el *Garbage Collector* decida limpiarlas. Para hacerlo, Java consume más RAM de la necesaria y, de vez en cuando, hace micro-pausas (*Stop-the-World*) para barrer la memoria, lo que puede provocar pequeños tirones (*stuttering*) en animaciones 3D o al hacer scroll.
    - En **Rust**, no existe el *Garbage Collector*. Gracias a su sistema de *Ownership* (propiedad), en el exacto microsegundo en que una página sale de tu caché de lectura, Rust libera esos megabytes de RAM de forma determinista y sin pausar el programa.
2. **Tiempo de arranque (Cold Start):**
    - Un binario de Rust arranca en milisegundos porque habla directamente con el sistema operativo. Java necesita levantar primero la Máquina Virtual (JVM) en memoria.

Rust vs. C / C++ (Empate técnico en velocidad)

Rust no es más rápido que C o C++; **están empatados** en el techo máximo de rendimiento porque ambos compilan directamente a instrucciones nativas del procesador (usando LLVM). La ventaja de Rust frente a C++ no es la velocidad, sino la **seguridad de memoria**: el compilador de Rust te impide cometer errores que en C++ causarían que el programa se cierre de golpe (*Segmentation Fault*) o tenga fugas de memoria al manipular imágenes.

3. ¿Significa esto que siempre hay que usar Rust?

No todo son ventajas. El precio que pagas por el rendimiento de Rust es:

- **Curva de aprendizaje y fricción:** El compilador de Rust (*Borrow Checker*) es muy estricto. Algo que en Python o TypeScript escribes en 5 minutos sin pensar en quién es "dueño" de una variable, en Rust te obligará a estructurar bien los datos.
- **Tiempo de compilación:** Compilar el proyecto tarda más segundos que ejecutar un script en Python o TypeScript.

Por eso la arquitectura híbrida (**Rust en el motor + TypeScript/React en la interfaz**) es tan popular hoy: usas Rust únicamente para los 4 archivos donde necesitas fuerza bruta (descomprimir el `.cbr`, redimensionar carátulas en paralelo con todos los núcleos de la CPU y servir los bytes) y haces todo el diseño visual y 3D cómodamente en TypeScript.

Sources

1. Rust vs Python: ¿Cuál Es Mejor Para Tu Proyecto? - Kinsta
2. Rust vs Python: Elegir la lengua adecuada para tu proyecto de datos
3. Mi experiencia con el rendimiento de Rust, comparado con Python

---

User prompt: Interfaz en rust no es posible?

Response: **Sí, es totalmente posible hacer la interfaz 100% en Rust**, eliminando por completo HTML, CSS, JavaScript, React y `npm`. Todo tu proyecto se compila con un único comando (`cargo run`) en un solo archivo ejecutable nativo.

Sin embargo, elegir entre **100% Rust** o **Rust + Web (Tauri)** tiene implicaciones directas en el diseño 3D que mencionaste y en la velocidad a la que programarás la interfaz.

1. Los mejores frameworks de interfaz gráfica (GUI) en Rust puro

Si decides no tocar nada de tecnologías web, estas son las herramientas líderes en el ecosistema Rust actual:

Framework	Estilo / Filosofía	Rendimiento	Qué tal lleva el 3D	Quién lo usa / Nivel
Iced	Inspirado en Elm (declarativo y reactivo). Renderiza directo en GPU con `wgpu`.	**Extremadamente alto**	Permite incrustar shaders y gráficos 3D (`wgpu` / Vulkan / Metal) dentro de widgets.	**System76** lo usa para programar el nuevo entorno de escritorio de Linux (*COSMIC*).
Slint	Tiene su propio lenguaje de marcado visual (`.slint`) muy parecido a QML/CSS.	**Instantáneo** (incluso en microcontroladores)	Bueno para transformaciones 2.5D, más limitado para escenas 3D complejas.	Muy usado en la industria automotriz y aplicaciones de escritorio limpias.
egui	*Immediate Mode GUI* (se redibuja a 60/120 FPS como un videojuego).	**Máximo**	Muy fácil de integrar con motores 3D como **Bevy** o `three-d`.	Herramientas técnicas, motores gráficos, editores de datos de alto rendimiento.
Dioxus	Sintaxis casi idéntica a React (`rsx!`), pero escrito en Rust.	**Alto**	Su renderizador nativo (*Blitz* / *Freya*) aún es más joven que el modo WebView.	Desarrolladores que vienen de React y quieren pasar a Rust sin cambiar de mentalidad.

2. Ejemplo real: Cómo se ve una interfaz en Rust puro (con `Iced`)

Para que veas la diferencia, así se programa una vista en **Iced** (cero JavaScript, 100% Rust):

```
use iced::widget::{button, column, image, row, text};
use iced::{Alignment, Element, Sandbox, Settings};

struct ComicReader {
    current_page: usize,
    total_pages: usize,
}

#[derive(Debug, Clone, Copy)]
enum Message {
    NextPage,
    PrevPage,
}

impl Sandbox for ComicReader {
    type Message = Message;

    fn new() -> Self {
        Self { current_page: 1, total_pages: 24 }
    }

    fn title(&self) -> String {
        String::from("Lector CBR - 100% Rust")
    }

    fn update(&mut self, message: Message) {
        match message {
            Message::NextPage => {
                if self.current_page < self.total_pages {
                    self.current_page += 1;
                }
            }
```

```
            Message::PrevPage => {
                if self.current_page > 1 {
                    self.current_page -= 1;
                }
            }
        }
    }

    fn view(&self) -> Element<Message> {
        let controls = row![
            button("Anterior").on_press(Message::PrevPage),
            text(format!("{} / {}", self.current_page, self.total_pages)),
            button("Siguiente").on_press(Message::NextPage),
        ]
        .spacing(20)
        .align_items(Alignment::Center);

        column![
            // Aquí iría el widget image("ruta_o_bytes_en_memoria")
            controls
        ]
        .align_items(Alignment::Center)
        .into()
    }
}
```

3. El gran dilema: ¿100% Rust vs. Tauri (Rust + React)?

Antes de tomar la decisión definitiva, pon en la balanza los pros y los contras reales para **tu caso concreto**:

Lo bueno de hacerlo 100% en Rust (`Iced` o `egui` + `wgpu`):

1. **Adiós definitivo a `npm` y `node_modules`:** Solo existe `Cargo.toml`. Un solo lenguaje para todo.
2. **Consumo ridículo de recursos:** La app consumirá unos 20–30 MB de RAM y arrancará en menos de 0.1 segundos, porque no tiene que levantar el navegador del sistema (*WebView*).
3. **Prestigio técnico ("Hardcore"):** Un lector de cómics con aceleración GPU nativa (`wgpu`) hecho 100% en Rust demuestra un dominio fuerte del lenguaje.

Lo malo (El coste oculto de una GUI en Rust puro):

1. **Hacer que se vea "bonito" cuesta el triple de trabajo:** En la web (React/Tailwind), poner sombras difuminadas, bordes redondeados, animaciones suaves, fuentes tipográficas y un carrusel 3D te lleva **15 líneas de CSS**. En Rust puro (`Iced` o `egui`), no existe el CSS completo: para hacer una carátula 3D que rote con sombras e iluminación tienes que programar un **shader en WGSL (WebGPU)** o configurar una cámara y una malla 3D a mano con matrices matemáticas.
2. **Ecosistema visual más pequeño:** No tienes miles de componentes prefabricados (modales animados, selectores, tooltips elegantes) listos para copiar y pegar como en React.

Mi recomendación honesta

- **Elige 100% Rust (`Iced` o `egui`)** si tu prioridad es aprender Rust a fondo, odias el ecosistema web (`npm`/JS) y te divierte la programación gráfica de bajo nivel (renderizado en GPU, shaders).
- **Elige Tauri (Rust + React/TS)** si quieres que la interfaz visual parezca diseñada por un estudio profesional con animaciones 3D fluidas en pocos días de trabajo, dedicando Rust solo a la lógica pesada.

¿Qué camino te motiva más: lanzarte al **100% Rust nativo** (con `Iced` o `egui`) o mantener la vía **híbrida con Tauri**?

User prompt: perdona rust mola pero por lo que he visto se necesitan muchisimas mas lineas de codigo. que opinas de Ruby?

Response: Tienes toda la razón con Rust: es un lenguaje muy verboso donde te pasas la mitad del tiempo peleando con el compilador para hacer cosas que en otros lenguajes resuelves en tres líneas.

Sobre **Ruby**: es un lenguaje precioso, súper expresivo y diseñado literalmente para la "felicidad del programador" (escribes poquísimas líneas y se lee casi como inglés). **Sin embargo, para una aplicación de escritorio con gráficos 3D y manejo de imágenes pesadas, Ruby es una de las peores opciones posibles hoy en día.**

1. ¿Por qué Ruby no encaja en este proyecto?

- **Está enfocado 99% en Backend Web (Ruby on Rails):** Casi todo su ecosistema vive en servidores web. Sus librerías para crear ventanas de escritorio (*Shoes*, bindings de *GTK* o *Tk*) están abandonadas, lucen como programas de Windows 98 o carecen de soporte para animaciones modernas y carátulas en 3D.
- **Empaquetado de escritorio casi imposible:** Distribuir una app de Ruby a un usuario de Windows o Mac en un `.exe` o `.dmg` sin obligarle a instalar Ruby por consola es un dolor de cabeza enorme (las herramientas antiguas como *Ocra* llevan años sin actualizarse bien).

- **Rendimiento y Concurrencia (GVL):** Al igual que Python, el intérprete estándar de Ruby bloquea los hilos reales de CPU. Al descomprimir un `.cbr` grande y generar miniaturas a la vez, la interfaz gráfica se congelaría.
- **Mercado laboral actual:** Aunque sigue habiendo mantenimiento en *Ruby on Rails*, para un portafolio moderno en GitHub de una app de escritorio, Ruby no te dará visibilidad frente a reclutadores.

2. Si buscas "Pocas líneas de código + Cero npm + App moderna", ¿cuáles son tus mejores opciones?

Si te gusta la limpieza de Ruby (escribir poco código, legible y directo) pero necesitas algo que cree aplicaciones de escritorio espectaculares con buen rendimiento y mucha demanda laboral, tienes tres candidatos ideales:

Lenguaje + Framework	Código necesario	Por qué te va a gustar (y brilla en GitHub)
1. Dart + Flutter	**Muy poco**	Es el rey para crear interfaces preciosas y animaciones 3D/2.5D en pocas líneas. **No usa `npm`**, compila a código nativo de Windows/Mac/Linux/Android, y una sola clase te dibuja toda la estantería con animaciones a 120 FPS.
2. Python + PySide6 (Qt / QML)	**Mínimo**	Python es igual de limpio y corto que Ruby. Usando **QtQuick / QML**, defines la interfaz y efectos 3D de forma declarativa en poquísimas líneas, y encima tienes todo el ecosistema nativo de IA/OCR de Python directo.
3. C# (.NET 9) + Avalonia UI	**Moderado-Bajo**	C# moderno ha eliminado casi todo el código repetitivo. Tiene `SharpCompress` (abre CBR y CBZ en 4 líneas de código) y un rendimiento altísimo sin la complejidad mental de Rust.

3. Comparativa real de líneas de código: Abrir un CBZ

Mira cuántas líneas necesitas en **Python** o **C#** frente a Rust para hacer exactamente lo mismo (listar las imágenes de un cómic):

En Python (5 líneas):

```
import zipfile

def abrir_comic(ruta):
    with zipfile.ZipFile(ruta, 'r') as z:
        return sorted([f for f in z.namelist() if f.lower().endswith(('.jpg', '.png', '.webp'))])
```

En C# con `SharpCompress` — abre tanto CBR (RAR) como CBZ (ZIP) automáticamente (6 líneas):

```
using SharpCompress.Archives;

public List<string> AbrirComic(string ruta) =>
    ArchiveFactory.Open(ruta).Entries
        .Where(e => !e.IsDirectory && e.Key.EndsWith(".jpg", StringComparison.OrdinalIgnoreCase))
        .Select(e => e.Key).OrderBy(k => k).ToList();
```

Mi consejo para que disfrutes programándolo y te quede un gran GitHub

Si descartamos Rust por ser demasiado largo y complejo, y quieres olvidarte de `npm`:

1. **Ve a por Flutter (Dart)** si tu prioridad es que la interfaz gráfica, las transiciones y las **carátulas 3D** se hagan en pocas líneas y se vean de película desde el primer día.
2. **Ve a por Python (con Qt/PySide6 o Flet)** si quieres escribir la menor cantidad de código posible y aprovechar que el OCR, la conexión con Ollama y el manejo de imágenes se escriben en un suspiro.
3. **Ve a por C# (.NET + Avalonia)** si buscas el estándar empresarial más robusto para escritorio en Windows/Linux/Mac donde leer archivos `.cbr` (RAR5) está resuelto de fábrica en una sola línea.

User prompt: en tema de rendimiento rust sigueu siendo el mejor no?

Response: **Sí, en rendimiento puro y bajo consumo de RAM, Rust sigue siendo el número uno indiscutible** (empatado técnicamente con C y C++). Ningún lenguaje con recolector de basura o intérprete puede ganarle en velocidad bruta ni en eficiencia de memoria.

Sin embargo, en ingeniería de software existe el concepto de **"rendimiento percibido por el usuario"**, y ahí es donde está el matiz importante para tu proyecto:

1. ¿Cuánta diferencia real notarás al pasar de página en un cómic?

Imagina que el usuario pulsa la flecha derecha para pasar a la siguiente página de un archivo `.cbz`:

Lenguaje	Tiempo en descomprimir y mostrar 1 página	¿Qué siente el ojo humano?	Consumo de RAM de la App
Rust	~3 milisegundos	**Instantáneo**	~30 MB
C# (.NET) / Dart (Flutter)	~8 a 12 milisegundos	**Instantáneo** (un parpadeo humano dura 100 ms)	~120 - 180 MB
Python	~15 a 30 milisegundos *(porque la librería `zipfile/Pillow` por debajo está escrita en C)*	**Casi instantáneo** (solo se nota si procesas 100 portadas de golpe)	~150 - 250 MB

La realidad práctica: Para abrir y leer un cómic, tanto Rust como C# o Flutter van a sentirse **igual de instantáneos** para el ojo humano, porque pasar de 3 ms a 10 ms es imperceptible.

Donde Rust aplasta a los demás es cuando el usuario añade una carpeta con **500 cómics de golpe**: Rust puede usar todos los núcleos del procesador para extraer las 500 portadas en un par de segundos sin apenas subir la RAM, mientras que en otros lenguajes tardará varios segundos más y el consumo de memoria subirá bastante.

2. La relación Esfuerzo vs. Rendimiento

Aquí es donde tienes que decidir qué valoras más al sentarte a programar:

```
RENDIMIENTO DE LA MÁQUINA
▲
│  [Rust / C++]  <-- Máximo rendimiento, pero escribes mucho código complejo.
│
│       [C# (.NET) / Flutter (Dart) / Go]  <-- EL PUNTO DULCE: 85% de la velocidad de Rust
│                                              con la mitad de líneas de código.
│
│                    [Python / Ruby] <-- Escribes poquísimo código, pero es más lento
│                                        y consume más recursos.
└──────────────────────────────────────────────► VELOCIDAD AL PROGRAMAR
```

3. ¿Cómo tener el rendimiento de Rust SIN escribir tanto código en Rust?

Si quieres que tu repositorio presuma de la velocidad de Rust pero no quieres escribir 2.000 líneas de Rust peleándote con el compilador, tienes dos caminos muy inteligentes:

1. **Volver a la idea de Tauri (pero simplificada):**
    - En lugar de hacer toda la app en Rust, **Rust solo tiene 1 archivo de 80 líneas** (abrir el ZIP/RAR y devolver la imagen). No tocas Rust para nada más.
    - Todo el resto de tu aplicación (la interfaz, el 3D, los botones, la llamada a Ollama, la base de datos) lo haces en TypeScript, donde escribes muy pocas líneas.
2. **Usar C# (.NET 9) o Flutter (Dart):**
    - Renuncias a ese último 15% de rendimiento extremo de Rust, pero a cambio tienes un lenguaje comodísimo, sin `npm`, compilado a código nativo muy rápido, y donde hacer toda la aplicación te llevará la tercera parte de tiempo y líneas de código.

¿Qué te tira más: **mantener ese pequeño motor de 80 líneas en Rust** para tener el sello de máximo rendimiento en GitHub, o prefieres **ir con un lenguaje cómodo como C# / Flutter** que sigue siendo muy rápido pero mucho más corto de escribir?

User prompt: todo con rust creo que mejor es un lenguaje en alza encima no?

Response: **Sí, totalmente.** Rust no solo lleva años siendo el lenguaje más admirado en las encuestas de Stack Overflow, sino que ya es un estándar industrial: está integrado en el kernel de Linux, Microsoft está reescribiendo partes críticas de Windows en Rust, y gigantes como AWS, Cloudflare o Discord lo usan para sus sistemas de máximo rendimiento. Tener un proyecto **100% en Rust** en tu GitHub envía un mensaje muy potente: demuestra que no te asustan los retos técnicos y que sabes construir software nativo ultrarrápido sin depender de capas web pesadas.

Además, el problema de las "muchas líneas de código" que viste antes tiene truco: el ejemplo anterior era con **Iced**, que te obliga a crear estructuras de mensajes para todo. Pero en Rust existen dos alternativas donde **escribes muchísimo menos código**:

Cómo hacer el proyecto 100% en Rust escribiendo la mitad de código

Para que no tengas que escribir código repetitivo (*boilerplate*) y puedas tener tanto interfaz moderna como **carátulas 3D**, tienes tres opciones dentro de Rust puro:

Framework en Rust	Código necesario	Cómo funciona	Por qué te conviene para tu lector
1. `egui` (+ `three-d` o `bevy`)	**Muy poco**	Modo inmediato: una botón es literalmente 1 línea de código (`if ui.button("Siguiente").clicked() { ... }`).	Es la forma más rápida y corta de programar interfaces en Rust puro. Además, se integra de lujo con motores 3D de Rust.

Framework en Rust	Código necesario	Cómo funciona	Por qué te conviene para tu lector
2. `Slint`	**Poco y muy limpio**	Separa el diseño visual en un archivo `.slint` (muy parecido a CSS) y la lógica en Rust.	No ensucias tu código de Rust con márgenes, colores ni tamaños de botones. Luce muy moderno de serie.
3. `Dioxus` (Modo Nativo)	**Moderado**	Usa macros para escribir la interfaz casi igual que si fuera HTML/React, pero 100% compilado en Rust.	Si te gusta cómo se estructuran las vistas web pero no quieres saber nada de `npm` ni de JavaScript.

Mira la diferencia real de código usando `egui` (100% Rust)

Olvídate de las 60 líneas del ejemplo anterior. Así se escribe la ventana completa de tu lector con **`egui` (`eframe`)**:

```
use eframe::egui;

struct MiLector {
    pagina_actual: usize,
    total_paginas: usize,
}

impl eframe::App for MiLector {
    fn update(&mut self, ctx: &egui::Context, _frame: &mut eframe::Frame) {
        egui::CentralPanel::default().show(ctx, |ui| {
            ui.heading("Mi Lector CBR en Rust");

            // Aquí se muestra la imagen de la página en 1 línea:
            // ui.image("file://portada.jpg");

            ui.horizontal(|ui| {
                if ui.button("⬅ Anterior").clicked() && self.pagina_actual > 1 {
                    self.pagina_actual -= 1;
                }

                ui.label(format!("Página {} de {}", self.pagina_actual, self.total_paginas));

                if ui.button("Siguiente ➡").clicked() && self.pagina_actual < self.total_paginas {
                    self.pagina_actual += 1;
                }
            });
        });
    }
}
```

Con solo eso ya tienes una ventana nativa acelerada por tarjeta gráfica (OpenGL / Vulkan / Metal / DirectX), sin `npm`, sin JavaScript y consumiendo apenas **25 MB de RAM**.

El Stack 100% Rust definitivo para tu repositorio

Si nos vamos con todo a Rust, estas son las piezas (*crates*) que pondremos en tu archivo `Cargo.toml` para cubrir todo lo que hemos hablado:

1. **Interfaz gráfica y 3D:**
   - **`eframe` / `egui` + `egui_extras`**: Para la interfaz rápida, paneles, buscador y carga de imágenes.
   - **`three-d`** o **`wgpu`**: Se incrusta dentro del panel de `egui` para renderizar los tomos y carátulas en **3D real** con iluminación y grosor según el número de páginas.
2. **Lectura de Cómics (CBR / CBZ):**
   - **`zip`**: Para abrir archivos `.cbz` en memoria.
   - **`unrar`**: Para abrir archivos `.cbr` (RAR4 y RAR5).
   - **`rayon`**: La joya de la corona de Rust. Con cambiar `.iter()` por `.par_iter()` (una sola palabra), Rust usará **todos los núcleos de tu procesador en paralelo** para generar las miniaturas de 100 cómics a la vez.
3. **Base de datos y Caché:**
   - **`rusqlite`**: SQLite empotrado directamente dentro de tu ejecutable `.exe` para guardar la biblioteca y por qué página vas leyendo.
4. **IA / OCR (Ollama y API Keys):**
   - **`reqwest` + `serde_json`**: Para mandar el recorte de la viñeta a tu Ollama local (`localhost:11434`) o a OpenAI/Gemini/OpenRouter.
